

《信息安全技术》期末考查报告

题目：基于 GitHub Actions 工作流的 CI/CD 安全风险检测复现与改进

课程名称：信息安全技术

选题方向：选题一，围绕 GitHub Actions workflow 安全检测进行论文复现与改进实验。

小组成员：郭翊涛（22308043）、负湘楠（20337264）

摘要

持续集成与持续部署已经成为现代软件工程的基础设施。GitHub Actions 让开源项目可以用 YAML 文件声明构建、测试、发布和部署流程，但它也把代码执行权限、仓库访问令牌、第三方 Action 和 Secrets 绑定在同一个自动化环境中。当 workflow 触发条件、权限声明或第三方依赖配置不当时，攻击者可能借助 pull request、未固定依赖、危险 shell 命令或泄露的环境变量影响 CI/CD 执行过程，并进一步造成供应链风险。本文选择 USENIX Security 2022 论文 Characterizing the Security of Github CI Workflows 作为复现对象。该论文提出了 CI/CD 平台应满足的安全性质，并对大规模 GitHub workflow 进行了实证分析。

本项目没有复现论文的大规模数据采集系统，而是围绕其核心思想实现了一个可运行的 GitHub Actions 静态安全检测工具。基础复现部分实现了对 pull_request_target、缺少 permissions、过高写权限、第三方 Action 未固定到完整 commit SHA、危险命令和可疑 Secret 使用方式的检测。改进部分进一步加入风险权重、综合评分、风险等级、修复建议和可视化输出，使结果不仅能指出是否存在问题，还能表达风险严重程度和修复优先级。实验使用 6 个 workflow 样例，其中包含自建安全样例、低中高风险样例和接近真实项目的公开场景样例。实验结果显示，工具能够区分无明显风险、中风险和高风险配置；其中 pull_request_target 与 checkout PR HEAD、write-all 权限和未固定第三方 Action 组合时风险评分最高，体现了 CI/CD 自动化配置中权限边界与代码来源控制的重要性。

一、研究背景、问题和动机

CI/CD 的出现改变了软件开发方式。开发者不再依赖人工在本地构建、测试和发布，而是把这些步骤写入自动化流水线，由平台在代码提交、分支合并、定时任务或人工触发时执行。这样的自动化可以提高交付效率，也能让测试和发布流程更稳定。但是，CI/CD 平台天然具有较高权限：它需要读取仓库代码、安装依赖、执行脚本、访问部署密钥、上传构建产物，有时还会把镜像或包发布到生产环境。一旦 CI/CD 配置被攻击者影响，攻击面就会从普通代码缺陷扩展到软件供应链。

GitHub Actions 是当前最常见的开源项目 CI/CD 平台之一。项目只需要在 .github/workflows 目录下放置 YAML 文件，就能声明 workflow 的触发事件、运行环境、作业、步骤和权限。每个步骤既可以执行 shell 命令，也可以引用第三方 Action。Action 类似软件依赖，封装了登录容器仓库、构建镜像、上传覆盖率、发布 release 等常见任务。便利性也带来新的安全问题：开发者可能直接使用 v3、main 等标签引用 Action，而不是固定到经过审计的 commit；workflow 可能默认继承比实际需要更高的 GITHUB_TOKEN 权限；pull_request_target 这类事件在处理外部贡献者代码时可能让不可信输入与高权限上下文交汇；Secrets 可能被写入日志或跨步骤环境变量；远程脚本可能通过 curl|bash 形式被直接执行。

本文关注的问题是：在课程项目可控规模内，如何复现论文关于 GitHub CI workflow 安全风险分析的核心思路，并在复现基础上做出具有实验价值的改进。完整论文分析了 447238 个 workflow 和 213854 个仓库，并比较 GitHub CI 与其他 CI/CD 平台的安全性质。课程作业受到时间、数据规模和网络条件限制，不适合重复论文的全量爬取与生态测量。因此，本项目把复现范围限定为静态规则检测：解析 workflow YAML，抽取触发器、权限、步骤、Action 引用和 Secret 使用方式，然后用安全规则判断风险。改进创新则聚焦在结果解释层面：引入风险评分和修复建议，使扫描结果能够体现不同配置之间的严重程度差异。

选择这一题目的动机有三点。第一，CI/CD 安全与信息安全技术课程中的访问控制、软件供应链安全、自动化执行环境隔离等主题高度相关。第二，GitHub Actions 配置文件结构清晰，适合用 Python 实现可复现的工程项目，不需要复杂硬件。第三，静态检测结果可以用 JSON、CSV 和图表呈现，便于验证改进方案是否提升了结果的可解释性和可比较性。

二、原论文介绍

原论文 *Characterizing the Security of Github CI Workflows* 发表在 31st USENIX Security Symposium，即 USENIX Security 2022。作者包括 Igibek Koishybayev、Aleksandr Nahapetyan、Raima Zachariah、Siddharth Muralee、Bradley Reaves、Alexandros Kapravelos 和 Aravind Machiry。USENIX Security 是信息安全领域的重要会议，论文时间和级别与本项目的研究范围相匹配。

论文的核心目标是研究 GitHub CI 生态中 workflow 使用方式是否满足安全要求。作者首先抽象出 CI/CD 系统应具备的四类安全性质：Admittance Control、Execution Control、Code Control 和 Access to Secrets。它们分别对应谁可以触发流水线、流水线是否会执行不可信代码、执行时使用的代码来源是否可控，以及 Secrets 是否可能被不可信代码访问。随后论文把 GitHub CI 与其他主流 CI/CD 平台进行比较，并对大规模公开仓库 workflow 进行测量，找出过高权限、pull request 触发下执行仓库代码、依赖第三方 Action 和使用存在安全问题 Action 等风险。

该论文的贡献在于把 CI/CD 安全问题从零散的最佳实践上升为系统化安全性质，并用大规模实证数据说明问题普遍存在。论文指出，workflow 的安全性并不只由单个配置字段决定，而是由触发器、权限、代码来源、第三方组件和 Secrets 共同决定。对于课程项目而言，这一点尤其重要：如果只检查某个 Action 是否未固定版本，工具只能给出孤立问题；如果把触发器、权限和步骤组合起来看，就能识别更接近真实攻击链的高风险配置。

三、原论文核心方案分析

从工程角度看，论文的分析流程可以拆分为五个步骤。第一步是收集公开 GitHub 仓库中的 workflow 文件。第二步是解析 YAML，提取 workflow 名称、触发事件、job、step、uses、run、permissions、Secrets 等字段。第三步是根据 CI/CD 安全性质构造判定条件，例如 workflow 是否由 pull request 触发、是否会 checkout 贡献者代码、是否拥有写权限、是否使用外部 Action。第四步是把规则应用到大规模数据集，统计不同风险的出现比例。第五步是结合攻击场景分析风险影响，并提出防御建议。

论文中的四类安全性质可以对应到本项目的检测规则。Admittance Control 强调触发入口，映射到 pull_request_target 和 pull_request 等事件检查。Execution Control 强调执行内容是否可由攻击者影响，映射到 checkout PR HEAD、执行来自事件上下文的 shell 命令和 curl|bash 等危险模式。Code Control 强调第三方代码来源是否可控，映射到 Action 是否固定到完整 commit SHA。Access to Secrets 强调敏感信息是否可能暴露，映射到 Secrets 是否被拼接进命令、写入环境文件或通过高权限上下文传递。

完整论文的检测对象非常大，并且包含对 GitHub 平台行为的逆向分析。本项目复现的是其中最适合同课程实验的静态规则部分。这样做的好处是实现透明、输入输出可复查、实验可重复；局限是无法检测所有运行时行为，也无法像论文一样得到生态级统计结论。

四、复现方案设计

本项目实现的工具位于 src 目录，入口文件为 scanner.py。工具接收单个 workflow 文件或包含多个 workflow 的目录，递归发现 .yaml 与 .yml 文件，随后调用 rules.py 进行规则检测，调用 risk_model.py 计算评分和等级，最后由 reporter.py 写出 JSON、CSV 和控制台报告，由 visualize.py 生成图表。

基础复现规则共有六类。第一，检测 pull_request_target 触发器。该触发器常用于在 PR 上执行带有目标仓库上下文的任务，如果它与 checkout 外部贡献者代码或 Secrets 使用结合，风险明显高于普通 pull_request。第二，检测是否缺少显式 permissions。显式最小权限是 GitHub Actions 安全实

践中的基本要求；如果 workflow 和 job 都没有 permissions，开发者很难确认 GITHUB_TOKEN 的实际权限边界。第三，检测 write-all 或 contents、actions、packages 等敏感作用域的 write 权限。第四，检测第三方 Action 是否未固定到完整 40 位 commit SHA。使用 tag 或 branch 虽然方便升级，但 Action 作者账号、仓库或发布流程一旦被攻击，引用者会自动执行变化后的代码。第五，检测 curl|bash、wget|sh、eval 和从事件上下文动态执行命令等危险 shell 模式。第六，检测 Secret 或环境变量可疑使用方式，例如把 token 写入 GITHUB_ENV 或在 echo、printf 中拼接 Secrets。

解析过程采用 PyYAML。由于 GitHub Actions 使用的 YAML 中 on 字段常常不加引号，而 PyYAML 在 YAML 1.1 语义下会把 on 识别为布尔值 True，本项目在 load_workflow 中加入兼容处理：如果发现 True 键且不存在字符串 on 键，就把 True 键还原为 on。这一处理虽然很小，但对实际扫描非常关键，否则触发器检测会失效。

具体算法可以描述为以下流程。输入阶段，工具首先判断用户给出的路径是单个文件还是目录；如果是目录，就递归查找所有后缀为 .yaml 或 .yml 的文件。解析阶段，工具将每个 YAML 文件转换为 Python 字典，并统一抽取 on、permissions、jobs、steps、uses、run、env 和 with 等字段。规则阶段，工具依次执行触发器规则、权限规则、Action 引用规则、命令规则和 Secret 使用规则。聚合阶段，工具把每条命中的风险转化为结构化 Finding，其中包含规则编号、标题、严重性、权重、位置、证据和修复建议。输出阶段，工具把所有 workflow 的 Finding 汇总为 JSON 和 CSV，并生成控制台报告与统计图表。

在规则设计上，本项目尽量避免只依赖关键字的粗糙判断，而是结合 workflow 结构进行定位。例如，权限规则既检查 workflow 顶层 permissions，也检查每个 job 自己的 permissions；Action 规则会跳过本地 Action 和官方 actions、github 命名空间下的 Action，只对第三方 Action 的 ref 做完整 SHA 判断；Secret 规则同时检查 run 命令和 step 级 env 字段；组合风险规则只在 pull_request_target 已经存在的前提下进一步检查 checkout 步骤。这样可以减少误报，并让输出位置更接近真实修复点。

从复现角度看，这些规则并不是对论文所有分析的完整替代，而是论文安全性质在课程规模中的具体化。论文关注的是 GitHub CI 生态的系统性风险，强调安全性质的缺失会在大量仓库中形成普遍攻击面；本项目关注的是单个项目提交前或审计时能否发现配置问题。二者的层次不同，但底层判断是一致的：CI/CD 安全不能只看 workflow 是否能成功运行，还要看谁触发了运行、运行的代码来自哪里、执行上下文有什么权限、敏感信息是否会被不可信步骤接触。

五、改进方案设计

相对基础规则检测，本文的改进方案有三个方面。

第一是风险评分。每条规则都被赋予权重：pull_request_target 权重为 5，缺少显式 permissions 权重为 3，过高写权限权重为 4，第三方 Action 未固定 SHA 权重为 3，危险 shell 命令权重为 4，可疑 Secret 使用权重为 3，pull_request_target 与 checkout PR HEAD 组合风险权重为 6。单个 workflow 的总分等于所有命中规则权重之和。评分 1 到 3 为低风险，4 到 7 为中风险，8 分及以上为高风险，0 分为无明显风险。

第二是修复建议。基础扫描器只告诉开发者“哪里有问题”，改进版还为每个发现生成推荐动作。例如缺少 permissions 时建议设置 permissions: contents: read；过高权限时建议只在必要 job 上单独授予 write；未固定第三方 Action 时建议固定到完整 commit SHA；危险命令时建议避免动态下载执行并校验来源；Secret 使用可疑时建议避免写入日志或跨步骤环境变量。

第三是可视化输出。工具生成 risk_distribution.png、risk_scores.png 和 rule_weights.png。风险类型分布图用于观察哪些规则最常命中，评分对比图用于体现不同 workflow 风险程度差异，权重表用于解释评分模型。这些图表让报告中的实验分析不只停留在文字描述，而是能直观展示复现方案和改进方案的差别。

评分模型的设计遵循两个原则。第一，能够直接形成攻击链的组合风险应当高于单点配置问题。例如 pull_request_target 与 checkout PR HEAD 同时出现时，风险不只是两个字段相加，而是意味着不可信贡献者代码可能进入较高权限上下文，因此 GHA007 被设置为最高权重。第二，影响范围大的权限

问题应当高于可局部修复的格式问题。例如 `packages: write` 可能是发布 workflow 的业务需要，但它一旦出现在普通测试 job 中就会扩大攻击后果，因此权重高于普通未固定版本。这样的权重并不表示漏洞一定会被利用，而是表示审计时的修复优先级。

修复建议也按最小权限和最小信任面设计。对于 `permissions`，建议显式设置最小权限，而不是简单删除字段；对于第三方 Action，建议固定完整 commit SHA，而不是只升级到较新 tag；对于危险命令，建议校验下载源和哈希，或把脚本纳入仓库并经过代码审查；对于 `Secrets`，建议减少跨步骤传递和日志暴露。这样写的原因是，安全工具如果只给出“禁止使用”的结论，往往无法落地；给出可替换的工程做法，才能体现改进方案的实用价值。

六、实验设计与运行环境

实验环境为 macOS，Python 版本使用本项目 `.venv` 虚拟环境。主要依赖包括 `PyYAML`、`matplotlib`、`pandas`、`reportlab`、`pypdf` 和 `pdfplumber`。运行方式如下：

```
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt
.venv/bin/python src/scanner.py data --json results/scan_results.json --csv results/scan_summary.csv --figures results/figures
```

实验数据包括 `data/sample_workflows` 下的 4 个自建样例和 `data/public_workflows` 下的 2 个典型公开场景样例。`secure_build.yml` 是安全基线样例，显式声明只读权限并固定官方 Action 到 commit SHA。`unpinned_action.yml` 缺少 `permissions` 且使用未固定第三方 Action。`dangerous_script.yml` 包含 `curl|bash` 和将 token 写入 `GITHUB_ENV` 的模式。`pull_request_target_risky.yml` 构造了高风险场景：`pull_request_target`、`write-all`、`checkout PR HEAD`、未固定第三方 Action 和动态命令执行同时出现。`typical_node_ci.yml` 模拟 Node 项目的 CI 配置，包含 `packages: write` 和 `codecov` Action。`container_publish.yml` 模拟容器发布工作流，包含包写权限以及 `docker` 系列第三方 Action。

实验步骤为：首先运行扫描器读取所有 workflow；其次记录每个文件命中的规则、证据、修复建议、风险评分和风险等级；再次将结果输出为 JSON 与 CSV；然后生成图表；最后把表格和图表写入报告，对典型样例进行案例分析。

为了保证实验可复现，本项目没有依赖实时访问 GitHub API。所有样例都保存在 `data` 目录中，扫描结果由本地命令生成。这样做牺牲了数据规模，但提高了实验稳定性，也便于教师或同学在离线环境中复查。实验关注的不是统计某个生态比例，而是验证规则是否能够识别预期风险、评分是否能够区分风险等级、输出是否能够支撑修复分析。因此，自建样例覆盖了每一类核心规则，公开场景样例则用于说明规则在较真实 workflow 中如何表现。

七、实验结果

7.1 样例扫描汇总

样例	命中数	评分	等级	规则
container_publish.yml	3	10	高风险	GHA003、GHA004、GHA004
typical_node_ci.yml	2	7	中风险	GHA003、GHA004
dangerous_script.yml	2	7	中风险	GHA005、GHA006
pull_request_target_risky.yml	5	22	高风险	GHA001、GHA003、GHA004、GHA005、GHA007
secure_build.yml	0	0	无明显风险	-
unpinned_action.yml	2	6	中风险	GHA002、GHA004

从汇总表可以看到，secure_build.yml 未命中任何规则，评分为 0。unpinned_action.yml 命中缺少显式 permissions 和第三方 Action 未固定 SHA，评分为 6，属于中风险。dangerous_script.yml 命中危险命令和 Secret 使用可疑，评分为 7，属于中风险。pull_request_target_risky.yml 命中 5 项规则，评分为 22，属于高风险。两个公开场景样例分别暴露出包写权限、第三方 Action 未固定等问题，其中 container_publish.yml 因两个 docker Action 均未固定到 commit SHA，评分达到 10。

7.2 风险类型分布

规则编号	风险类型	命中次数
GHA001	使用 pull_request_target 触发器	1
GHA002	缺少显式 permissions 配置	1
GHA003	存在写权限或过高权限	3
GHA004	第三方 Action 未固定到完整 commit SHA	5
GHA005	步骤中包含危险 shell 命令模式	2
GHA006	Secret 或环境变量使用方式可疑	1
GHA007	checkout 与 pull_request_target 组合风险	1

规则分布显示，GHA004 第三方 Action 未固定到完整 commit SHA 是最常见问题，共命中 5 次。GHA003 过高写权限命中 3 次。GHA005 危险命令命中 2 次。GHA001、GHA002、GHA006 和 GHA007 各命中 1 次。这与论文中的观察方向一致：CI/CD workflow 的风险往往不是孤立漏洞，而是由权限过宽、外部依赖和触发条件共同构成。

7.3 图表说明

图 1 risk_distribution.png 展示了各规则命中次数。图 2 risk_scores.png 展示了不同 workflow 的风险评分差异。图 3 rule_weights.png 展示了改进版评分模型的规则权重。由于评分模型把组合风险单独赋予高权重，pull_request_target_risky.yml 与其他样例形成明显差异，说明改进方案能把“高危攻击链”从普通配置问题中凸显出来。

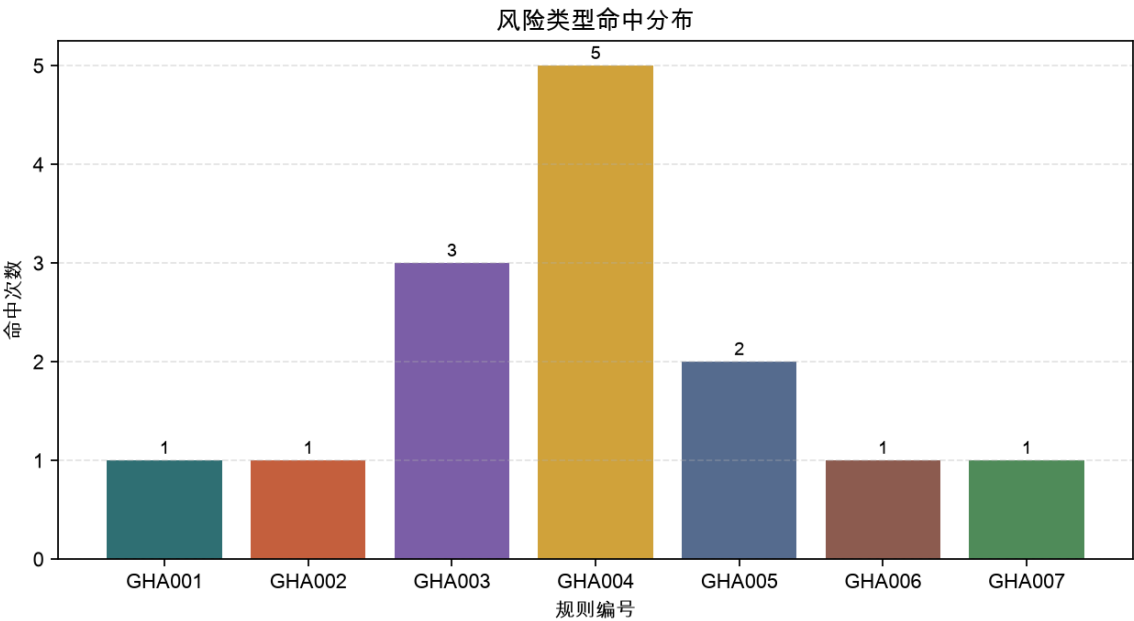


图 1 风险类型命中次数分布

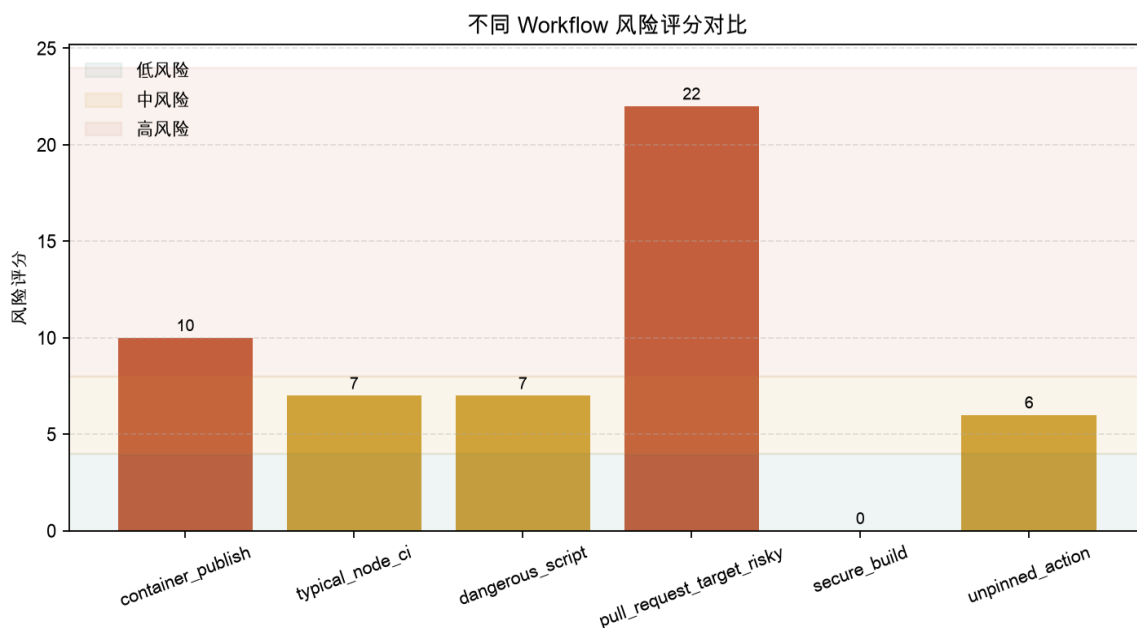


图 2 不同 workflow 风险评分对比

规则	风险项	严重性	权重
GHA001	使用 pull_request_target 触发器	high	5
GHA002	缺少显式 permissions 配置	medium	3
GHA003	存在写权限或过高权限	high	4
GHA004	第三方 Action 未固定到完整 commit SHA	medium	3
GHA005	步骤中包含危险 shell 命令模式	high	4
GHA006	Secret 或环境变量使用方式可疑	medium	3
GHA007	checkout 与 pull_request_target 组合风险	critical	6

图 3 规则权重设置

从数据解释角度看，未固定第三方 Action 是最频繁命中的风险，但它不一定总是最高优先级。原因是未固定 Action 的影响取决于 Action 权限、执行阶段和维护者可信度。如果 workflow 只在低权限测试场景执行，风险主要是构建结果被污染；如果 workflow 同时拥有发布权限或 Secrets，风险就会显著上升。相比之下，pull_request_target 与 checkout PR HEAD 的组合虽然只出现一次，但它直接体现了不可信输入、高权限上下文和代码执行之间的交叉，因此评分最高。这个结果说明，风险分析不能只看频率，还要看可利用条件和影响面。

7.4 典型案例分析

高风险样例 pull_request_target_risky.yml 最能体现 CI/CD 配置风险的组合效应。单独使用 pull_request_target 并不必然造成漏洞，因为很多项目会用它给 PR 打标签或做权限受控的元数据检查。但该样例同时使用 write-all、checkout PR HEAD、执行 pull request 标题构造的命令，并引用

未固定第三方 Action。此时攻击者只要提交一个 PR，就可能影响高权限工作流执行路径。虽然样例是人为构造，但它反映了真实安全分析中常见的风险链条：触发入口来自不可信贡献者，代码来源来自 PR，执行上下文具有目标仓库权限，敏感能力又通过 token 或 Secrets 暴露。

container_publish.yml 体现了发布类工作流的另一类风险。容器发布通常需要 packages: write，因此不能简单地把所有写权限都判为错误。改进版工具在报告中给出的是风险提示和修复建议，而不是机械阻断：建议将写权限限定在发布 job，并固定 docker/setup-buildx-action 与 docker/login-action 到完整 commit SHA。这样可以保留业务功能，同时减少 Action 被替换或供应链被污染时的影响范围。

dangerous_script.yml 的问题集中在运行命令层。curl|bash 形式把下载、信任和执行合并为一个不可观察步骤，既不校验脚本哈希，也不固定来源版本。如果脚本源被劫持或中间链路出现问题，CI 环境会直接执行攻击者代码。把 DEPLOY_TOKEN 写入 GITHUB_ENV 也会扩大 token 在后续步骤中的可见范围，不利于最小暴露原则。

secure_build.yml 作为安全基线也很重要。它证明工具不是简单地“看到 workflow 就报错”。该样例显式声明 contents: read，并把 checkout 与 setup-python 固定到完整 commit SHA，因此没有命中风险。实际项目中不可能完全消除 CI/CD 风险，但可以通过最小权限、固定依赖、避免动态执行和限制 Secrets 暴露来降低攻击面。安全样例给报告提供了对照组，使中高风险样例的评分差异更可信。

typical_node_ci.yml 的风险处在中间位置。它模拟常见 Node 项目，其中 packages: write 可能是为了发布包或缓存构建产物，codecov Action 也可能是正常质量分析流程。但从安全审计角度看，写权限和未固定第三方 Action 仍然值得关注。此类结果说明，扫描器输出不能被机械理解为“项目一定有漏洞”，而应作为配置审查清单：开发者需要判断写权限是否只授予必要 job，第三方 Action 是否可信，Secrets 是否只在必要步骤可见。

八、有效性与局限分析

本项目的有效性主要来自三方面。第一，规则来源与论文安全性质和 GitHub Actions 安全实践保持一致，覆盖了触发器、权限、代码来源和 Secrets 四个关键维度。第二，实验样例覆盖了无风险、中风险和高风险场景，能够验证评分模型是否具有区分度。第三，输出文件包含 JSON、CSV、图表和 PDF 报告，便于复查每一条发现的证据和建议。

局限也必须明确。首先，静态 YAML 分析无法知道运行时上下文。例如一个 workflow 可能通过 if 条件限制某些步骤只在可信分支执行，当前工具没有完整解释表达式语义，因此可能保守地提示风险。其次，工具无法判断某个第三方 Action 是否真的恶意或存在漏洞，只能判断它是否固定到不可变引用。再次，Secret 使用检测依赖模式匹配，能够发现明显的日志和环境变量暴露，但无法覆盖所有间接泄露方式。最后，风险评分来自人工权重，没有使用真实攻击数据训练，因此适合排序和教学分析，不适合作为唯一的合规结论。

尽管存在局限，静态检测仍然有现实意义。CI/CD workflow 是普通文本配置，变更频繁，很多问题可以在代码评审阶段被发现。如果把本工具集成到 pre-commit、pull request 检查或仓库安全审计中，就可以在 workflow 合并前提示开发者确认权限和依赖。与其等到供应链事件发生后追溯，不如在配置提交时就让风险显性化。

九、基础方案与改进方案对比

基础复现方案的价值在于能够发现 workflow 中的具体风险点，证明论文关于 GitHub CI 配置安全分析的核心思想可以被工程化实现。它的输入是 YAML 文件，输出是命中的规则与证据。基础方案简单、透明、可解释，适合定位问题。

改进方案的价值在于增强决策能力。首先，风险评分让多个 workflow 之间可以排序，便于确定修复优先级。其次，风险等级把原始规则命中转化为低、中、高风险表达，便于报告和管理场景使用。再次，修复建议把检测结果与整改动作连接起来，降低了工具输出与实际修复之间的距离。最后，图表让实验结果更容易被复核和展示。

二者的局限也不同。基础方案可能把所有问题看成同等严重，导致开发者难以区分“未固定一个低权限 Action”和“pull_request_target 执行不可信代码”之间的差别。改进方案虽然解决了排序问题，但权重仍是专家经验模型，不是通过大规模历史漏洞训练得到的概率模型。因此，本文把评分称为风险优先级，而不是精确漏洞概率。

十、总结与观点

通过本次复现与改进实验，我们完成了一个可运行的 GitHub Actions workflow 安全检测工具，并围绕论文提出的 CI/CD 安全性质实现了课程规模的实验验证。实验说明，静态规则检测虽然不能替代完整的动态沙箱或平台级权限隔离，但它能够在 workflow 提交前发现大量高价值配置问题。尤其是权限声明、Action 固定、危险命令和 Secrets 使用方式，都是开发者可以直接修复的风险点。

本文的观点是：CI/CD 安全不应被看作上线前的附加检查，而应被纳入软件供应链的默认开发流程。很多攻击并不需要突破应用运行时，而是通过自动化构建和发布环节进入供应链。GitHub Actions 这类平台越普及，workflow 文件就越接近“基础设施代码”。对基础设施代码进行审计、最小权限配置和依赖固定，是降低供应链风险的必要步骤。

本项目仍有改进空间。未来可以接入真实 GitHub 仓库采样，扩大数据规模；可以支持更多规则，例如 self-hosted runner 风险、workflow_run 事件风险、OIDC 配置风险和环境保护规则；也可以把静态规则与 CodeQL、Dependabot 或 OpenSSF Scorecard 结果结合，形成更完整的软件供应链安全评分。

参考文献

[1] Igibek Koishybayev, Aleksandr Nahapetyan, Raima Zachariah, Siddharth Muralee, Bradley Reaves, Alexandros Kapravelos, and Aravind Machiry. Characterizing the Security of Github CI Workflows. 31st USENIX Security Symposium, 2022.

<https://www.usenix.org/conference/usenixsecurity22/presentation/koishybayev>

[2] GitHub Docs. Secure use reference for GitHub Actions.

<https://docs.github.com/en/actions/reference/security/secure-use>

[3] GitHub Docs. Workflow syntax for GitHub Actions.

<https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax>

[4] GitHub Docs. Automatic token authentication.

<https://docs.github.com/en/actions/security-for-github-actions/security-guides/automatic-token-authentication>

[5] OpenSSF Scorecard. GitHub Actions security checks and software supply chain best practices. <https://github.com/ossf/scorecard>